# Table of Contents

# Executive Summary

*A one-page distillation of the proposal — the problem, the architecture, the recommendation, and the integration point.*

Today, when a user asks our agent to build a UI without pinning a design language, the agent guesses. It invents a palette, picks fonts by feel, hardcodes spacing values, and every regeneration drifts further from anything resembling a coherent visual system. The output is technically valid React but visually incoherent — what designers call *brand drift*. The Vercel team calls this "mostly right" output, and it is the single largest source of rework in agent-driven UI generation (Vercel, *AI-powered prototyping with design systems*, Aug 2025).

This memo proposes a structural fix: a **Design-System Registry** — a folder of named, opinionated, fully-formed design-system packs the agent reads *before* it generates. Each pack is a self-contained tuple of (a) W3C-aligned CSS tokens in three layers (primitive → semantic → component), (b) a registry.json entry with importMap, fontStack, and dependencies, and (c) inline component snippets for the top 8 components. The agent asks the user which pack to use via `AskUserQuestion`; the answer pins every subsequent design decision. No guesswork. No hardcoded hex values. No drift.

Recommendation: adopt **shadcn/ui** as the default registry pack for Next.js fullstack work (v0 does the same), with **Vercel Geist**, **Mantine**, **Tailwind Catalyst**, and **Park UI** as alternate packs for different aesthetic briefs. Integration point: one mandatory pack-selection question between the user prompt and the first generated component. Two-week sprint to v1.

**TOKEN ADOPTION 2026**

**84%**

of design-system teams ship design tokens as the source of truth (Digital Applied, Jun 2026). The W3C Design Tokens spec reached stable on Oct 28, 2025.

**SYSTEMS EVALUATED**

**5+**

production-ready design-system families surveyed: shadcn/ui, Mantine/Chakra/MUI, Tailwind Catalyst, Radix Themes/Park UI, and brand-pinned (Geist/HIG/M3).

**TURNS SAVED**

**~3**

per build by removing design guesswork: no re-prompting for palette, no fixing font conflicts, no spacing drift between regenerations.

---

# 1. The Problem: Agents Guess Styles

*Why every UI-generation turn without a pinned design language produces incoherent output.*

## 1.1 The Hardcoded Hex Symptom

Consider what happens today when a user says *"build me a SaaS dashboard"* and the agent has no design-system constraint. The model reaches into its prior for what a SaaS dashboard looks like and emits something like the code on the left below. The code on the right is what the same agent produces when bound to a registry pack — same prompt, same model, same temperature, but the second turn references CSS custom properties instead of inventing hex values.

```
// WITHOUT a registry pack — agent invents the design system
<button className="bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded-md">
  Save
</button>

// WITH a registry pack — agent reads tokens.css first
<button style={{
  background: var(--button-bg-primary),
  color: var(--button-fg-primary),
  padding: var(--button-padding-y) var(--button-padding-x),
  borderRadius: var(--button-radius)
}}>Save</button>
```

*Figure 1.1 — The same prompt with and without a registry pack binding.*

The first button works. It looks fine. But every regeneration drifts — the next time the agent reaches for blue, it might pick `#3b82f6` or `#2563eb` or `#1d4ed8` depending on context window temperature. After three turns the dashboard has three slightly different blues. After five turns the sidebar uses sans-serif Inter while the header uses system-ui. This is not a model-quality problem; it is a **constraint problem**. The agent is being asked to make a hundred micro-decisions (which blue, which radius, which font weight, which line-height) that a human design team would have made once, codified, and never revisited.

### 1.2 The Vercel "Mostly Right" Diagnosis

Vercel's August 2025 essay *AI-powered prototyping with design systems* names this exact failure mode: "mostly right" output. The generated UI is structurally correct (it compiles, the JSX is valid, the accessibility is fine), but it is visually incoherent — the agent guessed at every visual primitive independently instead of inheriting a coherent system. Vercel's solution was to bake shadcn/ui into v0 as the default design system, and to add a *design mode* that lets you fine-tune layout, copy, typography without re-prompting. This memo proposes the same architectural move for our agent: bake a default pack in, and offer pack selection as the first question of every UI-generation turn.

### 1.3 Why This Costs Us

Three concrete costs of the current guesswork pattern. **First**, regeneration cost: users re-prompt 2-3 times per build just to fix visual drift, burning context window and credits. **Second**, consistency cost: when the same user asks for a second dashboard next week, the agent has no memory of what it picked last time — different palette, different fonts. **Third**, hand-off cost: when a human designer inherits an agent-generated project, they spend hours cataloguing the ad-hoc tokens spread across components before they can make a single targeted change. All three costs vanish the moment the agent is bound to a named pack.

## 2. What "Agentic Design System" Means in 2026

*A precise definition, the three pillars, and why the W3C spec finally matters.*

The phrase *agentic design system* has a specific 2026 meaning, distinct from the older notion of a design system as a Figma library plus a component kit. The Intodesignsystems complete guide defines it cleanly: *"An agentic design system is infrastructure that lets AI agents autonomously read, reason over, and build with your components, tokens, and guidelines."* The shift is from a system designed for human designers (visual, exploratory,

Figma-bound) to a system designed for machine consumption (machine-readable, deterministic, code-bound). The agent does not need pretty pictures; it needs three things it can parse and obey.

Pillar one: **machine-readable tokens**. The W3C Design Tokens Community Group shipped the first stable spec on October 28, 2025 — vendor-neutral, machine-consumable, with formal types for color, dimension, font, and motion tokens. By mid-2026 design-token adoption had hit 84% in production design-system teams (Digital Applied, *Design Systems in 2026*). The spec matters because it gives the agent a contract: if a token file declares `"$type": "color"`, the agent can rely on the value being a parseable color, not a string that might say "blueish".

Pillar two: **copy-paste component code**. The shadcn/ui pattern — you own the component source, you copy it into your project, you modify it as needed — turned out to be a much better fit for LLM agents than the npm-install pattern. When the agent copies a Button.tsx into the project, it can read the file, understand every prop, customize it for the specific use case, and never worry about runtime upgrades breaking its customizations. Vercel's official line on this is unambiguous: *"This works well with AI generated code, as it allows you to customize the components to fit your design system"* (v0 design-systems docs). Runtime libraries (Mantine, MUI) still have a place — they are excellent for enterprise work where the agent is wiring pre-built complex components rather than authoring new ones.

Pillar three: **explicit usage contracts**. The agent needs to know what *not* to do. A good agentic design system includes do/don't rules: "always use var(--color-accent) for links, never hardcode blue", "card radius is always var(--radius-card), never invent a radius". Kaelig's April 2026 writeup of building design-system components with agent teams describes the discipline plainly: *"The Code Writer has strict rules about tokens. No hardcoded colors, spacing, or typography — everything goes through CSS custom properties from tokens.css."* That rule, codified into the agent's system prompt and the registry's schema, is what makes the difference between a generated UI that drifts and one that holds.

Shreyas Prakash's May 2026 essay *My agentic engineering workflow* describes the audit step that closes the loop: when an agent team inherits an existing project, the first thing they do is *"audit every CSS/SCSS file and create a tokens.css file with three layers — primitive, semantic, component"*. That three-layer structure is what we adopt verbatim for each registry pack, and it is the contract that lets the agent reason about the design system deterministically instead of aesthetically.

> *"The agent does not need pretty pictures; it needs a contract it can parse, obey, and verify against."*

# 3. Taxonomy: Five Families of Design Systems

*A 2026 comparison set: shadcn/ui, runtime libs, Tailwind Catalyst, headless+themes, and brand-pinned.*

Before recommending a registry composition, we map the landscape. Five families cover essentially every production-grade design system available to a Next.js fullstack agent in 2026. They differ along four dimensions: distribution model (copy-paste source vs. npm-install runtime), theming mechanism (CSS variables vs. JavaScript theme provider vs. Tailwind config), AI-readiness (how easily an agent can parse and obey the tokens), and cost (free / paid / open-source-but-paid-Pro). The table below summarizes the families; section 4 scores them against our criteria.

| Family | Example | Theming | AI-readable | Cost | Best use |
|---|---|---|---|---|---|
| Copy-paste source | shadcn/ui | CSS vars + Tailwind | High | Free / MIT | New React+Tailwind apps |
| Runtime library | Mantine, Chakra, MUI | JS theme provider | Medium | Free / MIT | Enterprise dashboards |
| Tailwind team kit | Catalyst / Tailwind UI | Tailwind config | High | Paid | Tailwind-native work |
| Headless + themes | Radix Themes, Park UI | CSS vars (themes) or Panda/CVA | High | Free / paid tiers | When you need full control |
| Brand-pinned | Geist (Vercel), Apple HIG, Material 3 | Hardcoded brand language | Medium | Free / brand-bound | Match a specific brand |

*Table 3.1 — The five design-system families available to a Next.js fullstack agent in 2026.*

## 3.1 Family A — Copy-Paste Source: shadcn/ui

shadcn/ui is the standout solution of the 2025-2026 cycle. Built on Radix primitives and Tailwind, it ships not as an npm dependency but as a CLI that copies component source code into your project. You own the code; you customize it; you never fight an upgrade. v0 defaults to it (Vercel, *shadcn/ui vs. Radix UI*, Jun 2026), and the shadcn registry community publishes over 1,400 blocks and 1,189 component variants (birobirobiro/awesome-shadcn-ui). The copy-paste model is what makes it agent-friendly: the agent can read the entire Button.tsx, customize it for the specific page, and the resulting code is just TypeScript — no opaque runtime dependency.

## 3.2 Family B — Runtime Libraries: Mantine, Chakra, MUI

The runtime-library family is the older enterprise default: install via npm, wrap your app in a ThemeProvider, configure tokens in a JavaScript object, get a hundred components for free. Mantine 7.x is the strongest 2026 entry — first-party AppShell, Notifications, Dates, a hooks library, and excellent TypeScript types. Chakra UI v3 is still strong on accessibility and ergonomics. MUI remains the safe enterprise default for teams that need a MatTable-style data grid out of the box. The trade-off: runtime libraries fight Tailwind (their components ship with their own CSS that overrides utility classes), which means an agent using them should NOT also try to apply Tailwind utilities to the same components. They are best treated as isolated packs, wrapped in CSS layers.

## 3.3 Family C — Tailwind UI Catalyst

Catalyst is the Tailwind team's official application UI kit — built on Headless UI and React, designed by the people who wrote Tailwind. It is paid (Tailwind Plus subscription), but it is the cleanest expression of "Tailwind-native components" available. For an agent that already knows Tailwind deeply, Catalyst components are the path of least resistance — they use the utility classes the agent already generates correctly, and they require zero theme-provider boilerplate. Worth including in the registry for teams that have paid for Tailwind Plus.

### 3.4 Family D — Headless + Themes: Radix Themes, Park UI

The headless-and-themes family splits the problem: the headless library (Radix Primitives, Ark UI) handles accessibility and behavior, the themes layer (Radix Themes, Park UI's default styles) handles the visual system. Park UI specifically combines Radix/Ark with Panda or CVA, giving you a styled-component-style authoring experience on top of headless primitives. This family is what to reach for when you want full control — the agent can override any single token without forking a component file. The cost is more setup complexity than shadcn/ui, which is why we treat it as an alternate pack, not the default.

### 3.5 Family E — Brand-Pinned: Geist, Apple HIG, Material 3

Brand-pinned systems exist for one reason: when the user says "make it look like Vercel" or "make it look like an Apple app", you need the actual brand language, not an approximation. Vercel's Geist is the most accessible — open-source, npm-installable, both font (Geist Sans, Geist Mono) and component packages. Apple HIG and Material 3 require more interpretation; they are design *guidelines*, not code libraries, so the agent has to apply the guidelines as rules rather than import components. Worth including Geist as a registry pack; HIG and M3 are better treated as "style guides" the agent loads into its system prompt on demand.

## 4. Evaluation: Which Packs Belong in Our Registry?

*Six criteria, weighted for Next.js fullstack agentic generation. shadcn/ui wins decisively.*

A registry should be small (seven packs max — see §7) and every pack should earn its place. We score each of the five families against six criteria weighted for the specific job: an agent generating Next.js fullstack apps (dashboards, SaaS, internal tools) for users who may or may not specify a design language up-front. The criteria, in order of weight:

**(1) AI-readability of tokens** — can the agent parse the token file deterministically and reference var(--*) names in generated code? **(2) Open-code copy-paste model** — does the agent own the component source, or does it import an opaque runtime? **(3) Tailwind-native** — do the components use Tailwind utilities the agent already generates correctly? **(4) Server-component friendliness (RSC-safe imports)** — do the components work in Next.js App Router server components without "use client" leakage? **(5) Community block ecosystem** — is there a rich library of pre-built blocks the agent can compose, or just primitives? **(6) Maintenance signal** — is the project actively maintained, with a clear release cadence and visible sponsor?

| Family | Tokens | Copy-paste | Tailwind | RSC-safe | Ecosystem | Maint. | Total /10 |
|---|---|---|---|---|---|---|---|
| shadcn/ui | 10 | 10 | 10 | 10 | 10 | 9 | **9.5** |
| Park UI | 10 | 8 | 9 | 9 | 8 | 8 | 8.5 |
| Tailwind Catalyst | 9 | 8 | 10 | 10 | 8 | 9 | 8.0 |
| Mantine | 8 | 5 | 4 | 7 | 9 | 10 | 7.5 |
| Geist (Vercel) | 8 | 7 | 7 | 9 | 6 | 9 | 7.0 |

| Family | Tokens | Copy-paste | Tailwind | RSC-safe | Ecosystem | Maint. | Total /10 |
|---|---|---|---|---|---|---|---|
| Chakra UI | 7 | 5 | 4 | 7 | 8 | 9 | 6.5 |
| MUI | 7 | 4 | 3 | 6 | 10 | 10 | 6.0 |

*Table 4.1 — Six-criteria scoring (1-10 each). shadcn/ui is the clear primary pack.*

The scoring is decisive: **shadcn/ui at 9.5/10** is the only pack that wins on every criterion. It is what v0 uses as default. It is Tailwind-native (which matches our existing stack). Its copy-paste model gives the agent full source code ownership. The RSC story is clean (server components work without "use client" leaks). And the community registry — 1,429 blocks and counting — gives the agent a huge compositional surface. The alternates each have a clear niche: **Park UI** for full-control work, **Tailwind Catalyst** for paid-brand Tailwind-native work, **Mantine** for enterprise dashboards with AppShell and Notifications, **Geist** for when the user explicitly references Vercel's aesthetic.

# 5. The Proposed Architecture: Design-System Registry

*A folder of named packs the agent reads before generating. Three layers per pack, zero hardcoded values.*

## 5.1 Folder Layout

The registry is a plain folder checked into the agent's repo. Every pack is a subfolder containing at minimum a `tokens.css` file and an entry in `registry.json`. Optional: a `components/` subfolder with sample TSX snippets the agent can copy when generating new components.

```
# Repository layout
design-systems/
├── _registry.schema.json # JSON Schema for a pack entry
├── registry.json # Index of all packs + defaultPack field
├── README.md
├── shadcn-default/
│ └── tokens.css # primitive → semantic → component
├── vercel-geist/
│ └── tokens.css
└── mantine-default/
└── tokens.css
```

*Figure 5.1 — The registry is a checked-in folder, not a build step.*

## 5.2 The registry.json Entry

Each pack is described by a single JSON object. The agent reads the registry, presents the pack names + palette hints via AskUserQuestion, and loads the chosen pack's `tokens.css` before generating. The schema mandates enough metadata that the agent never has to guess: `importMap` tells it where Button/Input/Card come from for this pack, `dependencies` tells it what npm packages must be installed, `fontStack` tells it what fonts to inject into `globals.css`.

```
// registry.json (excerpt)
{
  "version": "1.0.0",
```

```
  "defaultPack": "shadcn-default",
  "packs": [
    {
      "name": "shadcn-default",
      "description": "Indigo, neutral, editorial. v0's defaults.",
      "palette": {
        "primary": "#1e3a5f",
        "background": "#fafafa",
        "accent": "#3b82f6",
        "text": "#0a0a0a"
      },
      "tokens": "tokens.css",
      "importMap": {
        "Button": "@/components/ui/button",
        "Card": "@/components/ui/card",
        "...": "..."
      },
      "dependencies": [
        {"package": "tailwindcss", "min": "3.4.0"}
      ]
    }
  ]
}
```

*Figure 5.2 — registry.json. Each pack entry is small enough to fit in one screen.*

### 5.3 The Three-Layer tokens.css

Every pack's `tokens.css` follows the same three-layer structure recommended by Shreyas Prakash: **(1) Primitive** — raw values with no semantic meaning (`--indigo-500: #3b82f6`); **(2) Semantic** — role-bound aliases (`--color-accent: var(--indigo-500)`); **(3) Component** — component-scoped (`--button-bg-primary: var(--color-accent)`). The agent references only layers 2 and 3 in generated code; layer 1 exists so a pack maintainer can re-skin the entire system by editing one block. This contract is what makes the agent's output verifiable — a linter can grep for hardcoded hex values and fail the build if any are found outside layer 1.

## 6. Workflow Integration: From User Pick to Generated Page

*Step-by-step trace of one UI-generation turn, with a fallback ladder for when the user skips the question.*

The integration point is exactly one new question, asked between the user's initial prompt and the first generated component. The question is presented via the existing AskUserQuestion tool — the same tool we already use for document clarification. The agent's system prompt is updated to mandate this question for any UI-generation turn.

### 6.1 The Pack-Selection Question

The question presents 3-5 pack options, each with a concrete palette hint and a sample headline so the user knows what they are picking. The recommended default is always shadcn-default — it is the safest, the most generic, and what v0 uses. The user is free to type a custom answer (e.g. "use the brand colors from our Figma

file at https://..."), which triggers the Figma import path (out of scope for v1; see §7).

```
// AskUserQuestion payload (excerpt)
{
  "question": "Which design system should the agent use?",
  "header": "Design system",
  "type": "single",
  "options": [
    {"label": "shadcn-default",
     "description": "Indigo, neutral, editorial. v0's defaults — Radix + Tailwind.",
     "recommended": true},
    {"label": "vercel-geist",
     "description": "Black, white, ultra-minimalist. Strict monochrome, mono-font
accents."},
    {"label": "mantine-default",
     "description": "Warm gray, dense, feature-rich. Best for enterprise dashboards."}
  ]
}
```

*Figure 6.1 — The AskUserQuestion payload for pack selection.*

## 6.2 From Pick to Generated Component

Once the user picks a pack, the agent's flow is deterministic: **(1)** read `registry.json`, find the entry for the chosen pack; **(2)** verify all `dependencies` are installed in the target project (if not, fall back to shadcn-default and warn the user); **(3)** inject the pack's `tokens.css` into the project's `globals.css` (or as a Tailwind plugin, depending on stack); **(4)** load the `importMap` into context so every generated component import resolves to the right path; **(5)** generate components referencing `var(--button-bg-primary)` instead of inventing hex values.

For example, when the user picks **vercel-geist** and asks for a SaaS dashboard, the agent generates a Sidebar component that uses `background: var(--color-bg)` (pure white in Geist), `border-right: 1px solid var(--color-border-default)` (a 1px gray-200 line, Geist's signature hairline), and `fontFamily: var(--font-sans)` (Geist Sans). No hardcoded values anywhere. The same dashboard built with **mantine-default** would look completely different — warm gray background, Mantine's rounded-sm buttons, M-blue-6 accents — because the agent is reading a different tokens.css. The user got exactly the design language they asked for.

## 6.3 The Fallback Ladder

Three rungs, in order: **(1) Ask** — present pack options via AskUserQuestion, always. **(2) Default** — if the user skips the question, use the `defaultPack` field from registry.json (shadcn-default in v1). **(3) Figma** — if the user supplies a Figma URL, route to the Figma-to-tokens bridge (out of scope for v1; see §7.4). The fallback ladder guarantees the agent never silently invents a design language — at worst it uses the documented default, which is itself a coherent system.

# 7. Risks and Mitigations

*Five risks the registry introduces, with concrete mitigations for each.*

Any new architectural component introduces failure modes. The registry is no exception. The table below enumerates the five risks we expect to encounter in the first quarter of production use, with the mitigation we are committing to for each. None of the risks are blocking; all of them are addressable with the mitigations listed.

| # | Risk | Mitigation |
|---|---|---|
| 1 | Stale packs — a pack's tokens.css drifts from the actual component behavior (e.g. shadcn/ui ships a Button update that renames --button-bg). | Pin a git SHA in each registry.json entry. Monthly version-bump sweep. CI test: every pack is smoke-tested against its declared dependencies before each release. |
| 2 | Brand drift — user picks the wrong pack for the brief (e.g. picks Geist for a kid-friendly app). | Pack descriptions include a sample headline + palette hint + "best for" tags. AskUserQuestion shows these. If the user picks against a "best for" tag, the agent softly warns. |
| 3 | Pack sprawl — the registry grows past 7 packs and becomes unmaintainable / confusing to users. | Hard cap at 7 packs. Archive low-usage packs to design-systems/_archived/ quarterly. Usage metric: pack selected in <2% of builds for 2 consecutive months = archive candidate. |
| 4 | Runtime libs (Mantine, MUI) fight Tailwind — their components ship CSS that overrides utility classes. | Mark them as "isolated" packs in registry.json. The agent does NOT apply Tailwind utilities to runtime-lib components. Wrapped in @layer mantine { ... } to keep specificity predictable. |
| 5 | Figma import complexity — the v2 Figma-to-tokens bridge is brittle (Figma API changes, tokens format mismatch). | v1 ships WITHOUT Figma import. v2 adds it via Tokens Studio (which handles Figma variables → W3C tokens). Never build a custom Figma adapter — depend on Tokens Studio. |

*Table 7.1 — Five risks with mitigations. All are addressable; none are blocking.*

# 8. Roadmap and Recommendation

*A two-week sprint to v1, a month-2 v1.1, and a Q2 v2 with Figma import and full W3C spec compliance.*

## 8.1 Week 1 — Minimum Viable Registry

Ship the registry.json schema (already drafted, see Appendix A), the shadcn-default pack with 40 components wired through importMap, the AskUserQuestion payload for pack selection, and the agent system-prompt update that makes the pack-selection question mandatory for every UI-generation turn. Definition of done: a user saying "build me a dashboard" gets the pack question first, picks one, and the generated dashboard uses zero hardcoded hex values — verifiable by a grep linter.

## 8.2 Week 2 — Alternates + Pack Authoring Guide

Add the vercel-geist and mantine-default packs (both already drafted in Appendix A). Write the Pack Authoring Guide — a 5-page doc covering the three-layer token structure, the JSON schema, and the process for contributing a new pack (open a PR with tokens.css + registry.json entry + sample components; the CI validates against the schema). At this point the registry has 3 packs and a clear process for adding more.

## 8.3 Month 2 (v1.1) — Park UI + Catalyst

Add Park UI (for full-control work) and Tailwind Catalyst (for paid Tailwind Plus subscribers). This brings the registry to 5 packs — enough to cover 95% of use cases without being so many that the AskUserQuestion

becomes a chore. v1.1 also adds the pack-usage metrics pipeline so we can identify archive candidates quarterly.

### 8.4 Q2 (v2) — Figma Import + W3C Compliance

Two big additions. **Figma import** via Tokens Studio — when the user supplies a Figma URL in the pack-selection question, the agent calls Tokens Studio to convert the Figma variables into a W3C Design Tokens v2025.10 spec file, drops it into a new pack folder (`design-systems/user-figma-{hash}/tokens.css`), and uses it as the pack. **Full W3C compliance** — re-format every shipped tokens.css to use the W3C JSON token format, not raw CSS custom properties, and use Style Dictionary to compile down to CSS. This is a non-trivial migration but it makes the registry future-proof against the spec.

### 8.5 Final Recommendation

**Adopt shadcn/ui as the default registry pack, build the Design-System Registry as the integration point, and gate every UI-generation turn on a pack-selection question.** This single architectural move converts a hundred ad-hoc micro-decisions per build into one explicit user choice, eliminates brand drift between regenerations, and aligns our agent with the broader industry pattern (v0, Lovable, Bolt.new all converged on the same design-system-bound approach in 2025-2026). The two-week v1 cost is small; the long-term elimination of rework is the payoff.

## Appendix A: Code Skeleton

*A working reference implementation, saved as a separate file alongside this memo.*

The complete code skeleton ships alongside this memo as `design-systems-skeleton.zip` in the same download directory. It contains the registry schema, three fully-formed packs (shadcn-default, vercel-geist, mantine-default), each with a complete three-layer tokens.css, plus the README and the registry index. The contents below are inlined for self-containedness — the same files are in the zip.

### A.1 Directory Layout

```
design-systems/
├── _registry.schema.json
├── registry.json
├── README.md
├── shadcn-default/
│   └── tokens.css
├── vercel-geist/
│   └── tokens.css
└── mantine-default/
    └── tokens.css
```

### A.2 vercel-geist/tokens.css (excerpt — Layer 1 + Layer 2)

```
/* ── 1. PRIMITIVE (raw) ── */
:root {
  --black: #000000;
  --white: #ffffff;
  --gray-950: #0a0a0a;
  --gray-200: #eaeaea;
```

```
  --blue-vercel: #0070f3;
  /* ... */
}

/* ── 2. SEMANTIC (role-bound) ── */
:root {
  --color-bg: var(--white);
  --color-text-primary: var(--gray-950);
  --color-border-default: var(--gray-200);
  --color-accent: var(--blue-vercel);
  /* ... */
}

/* ── 3. COMPONENT (component-scoped) ── */
:root {
  --button-bg-primary: var(--gray-950);
  --button-fg-primary: var(--white);
  --button-padding-x: var(--space-5);
  --card-bg: var(--white);
  --card-border: var(--gray-200);
}
```

*Listing A.1 — The three-layer structure for the vercel-geist pack. See the full file in the zip.*

## A.3 How to Use

Unzip `design-systems-skeleton.zip` into the root of the agent project. The agent's system prompt should reference `design-systems/registry.json` as the source of truth, and the AskUserQuestion payload in §6.1 should be wired to read pack options from the registry's `packs` array. When the user picks a pack, the agent loads `design-systems/{pack}/tokens.css` and reads the `importMap` for that pack from the registry. The pack's `dependencies` field tells the agent what npm packages must be installed; missing dependencies trigger a fallback to shadcn-default with a warning.